

STEAMing DEV GOSSIP

WEB DESIGN PROCESS BOOK

Project Overview

The video game industry has become one of the most competitive sectors of the entertainment world, with more than 225'000 digital games across a wide range of genres and budgets released on all platforms in 2025, of which 10% are PC games. Of these, more than 95% of titles were released on Steam, Valve's dominant digital storefront, where a game's performance can be decisive for its commercial success. Yet with such a massive and ever-growing catalog, identifying what truly sets successful games apart is far from straightforward, and developers often lack the tools to make sense of these complex patterns at a glance.

Our project aims to help game developers and publishers to better understand what makes a game successful on the Steam platform. To do so, we have scrapped a large amount of data from the Steam store, and we have built a web application that allows us to explore this dataset through several complementary visualizations. Users can inspect individual games and their attributes, explore top rankings across various categories, analyze relationships between multiple game attributes simultaneously through a parallel coordinate plot, and observe how the number of releases per genre has evolved over time through an animated chart race, reflecting the shifting dynamics of the game market.

Early Design Decisions

- **Motivations:**

As prospective game designers, our group was leaning towards game statistics very early on in the project. We wanted to have tools that could allow us in the future to have a better idea of the game market and what makes a game popular. As the biggest indie game platform, Steam was the perfect subject for our project.

- **Inspiration:**

Thankfully for us, a huge database already existed for Steam, called SteamDB. Although it's a project that wasn't welcoming further collaborations, we were

greatly inspired by how their website was organized and their scrapping methods.

- **Up-to-date Scrapping:**

We opted to implement an up-to-date scrapping method to be able to see more recent releases (since our dataset stops in April 2025). We were, however, unable to include this more recent scrap in our project because it would have taken too long, but all the commands and packages needed are included in the project, so it is possible to perform the scrapping by ourselves.

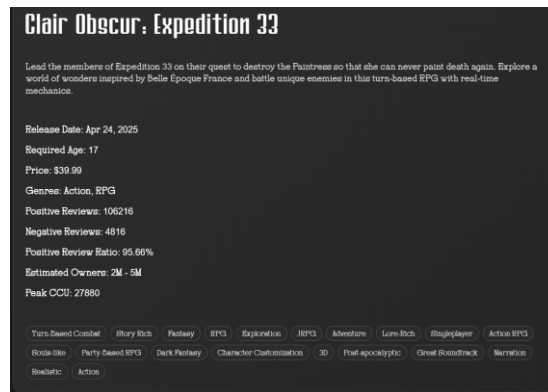
Dataset Summary

Our dataset contains data on approximately 150'000 games. Each entry contains many attributes such as a game's name, release date, genres, price, positive and negative ratings, and many more. The dataset can be obtained and updated using the "Steam Game Scraper" developed by Martin Bustos, which is well integrated in the project repository. The raw dataset is stored as a heavy JSON file.

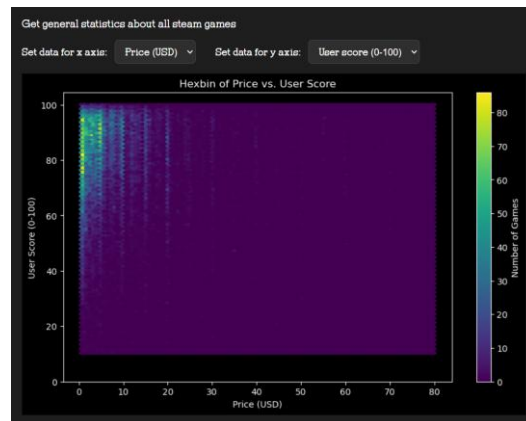
To make it more manageable, we developed a preprocessing pipeline which transforms the raw JSON file into a more compact format. The preprocessing pipeline loads the JSON file, can (but does not) filter out the games that do not satisfy specified criteria, removes unused attributes, and finally stores the resulting dataset in a MessagePack file, which is a more compact binary format. The processed dataset is about 10 times smaller than the raw Json file and can be loaded much faster (but still takes too much time to load in practice).

With this dataset, we can perform various analyses, mostly related to the relationship between prices, ratings, genres, and size of the playing community. This allows us to make informative visualizations that answer our project's goal.

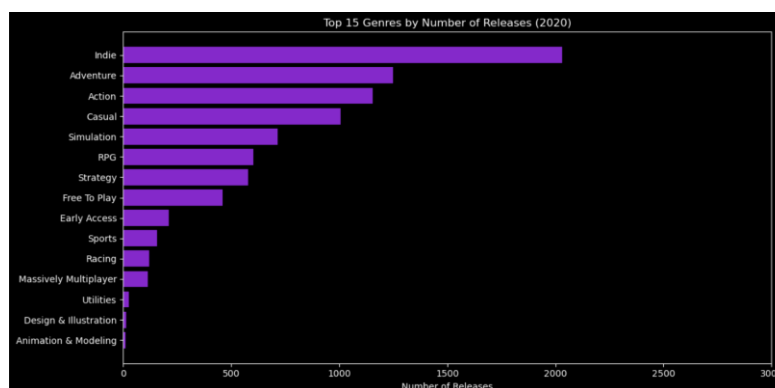
Visualization Design and Prototyping



Prototype for the "Game Insights" visualization



Prototype for the "Overview" visualization



Prototype for the "Developing Trends" visualization

For our website, the minimum viable product (MVP) was to have three main visualizations:

- A single game view, which allows the inspection of a single game's attributes, giving a detailed overview of its performances and characteristics.
- One or more visualizations showing an overview of the Steam catalog. This part was intentionally left vague, as we wanted to experiment with different types of visualizations to find the one that best suits our dataset and our project's goal. After a lot of prototyping, we ended up with one plot type which was particularly effective to explore the relationships between multiple game attributes simultaneously: the parallel coordinates plot.
- An animated chart race showing the evolution of the number of releases per genre over time. This visualization demonstrates how the dynamics of the game market has changed over time and how different genres have risen and fallen in popularity.

At the latest stages of the project, we also added a top-ranking section, which allows us to explore the top games across various metrics, such as the number of positive ratings, the price, the size of the playing community, and many more.

Website Implementation

The website is implemented using the D3.js library, which is a powerful tool for creating interactive data visualizations on the web. To guarantee a long-term availability of the website, we have implemented it as a static website, which can be easily hosted on GitHub Pages without the need for a backend server. The data is loaded directly from the MessagePack file, which is stored in the repository and can be updated regularly using the scrapper and the preprocessing pipeline.

As our website source code manages heterogeneous data, we felt the need to use a stricter type system to avoid bugs and make the code more maintainable. We therefore implemented the website using TypeScript, which is a superset of JavaScript that adds static typing and other features to the language. This allows us to catch errors at compile time and improve the overall quality of our codebase. The structure of the website is simple: it consists of a single HTML page that contains several sections, each dedicated to a specific visualization. Each visualization is implemented as a separate TypeScript file and is then injected into the HTML page at load time.

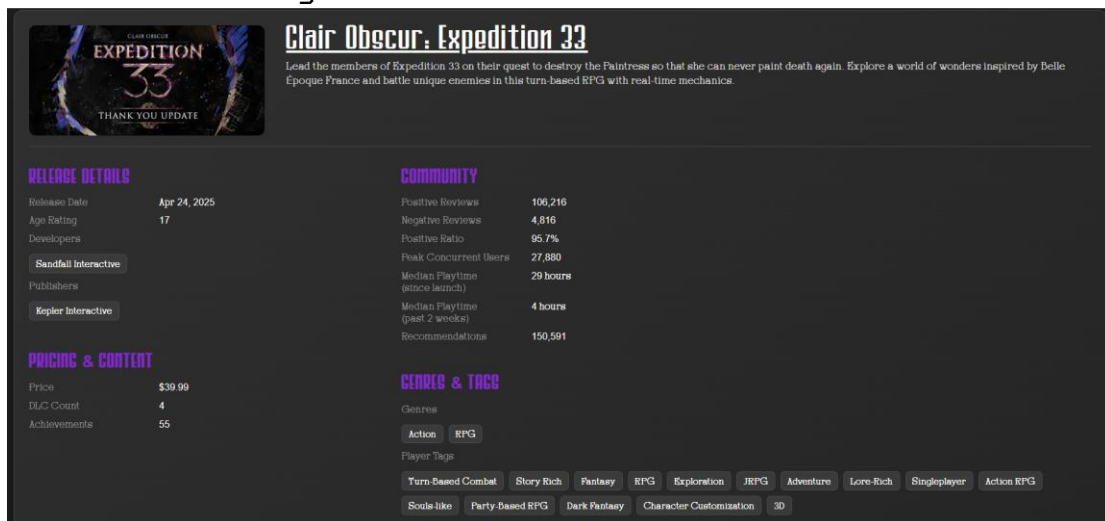
The Vite build tool is used to bundle and optimize the TypeScript code, and to generate the final JavaScript files that can be loaded by the browser. The project

environment also includes the gh-pages package, which allows us to easily deploy the website on GitHub Pages with a single command.

UI Components

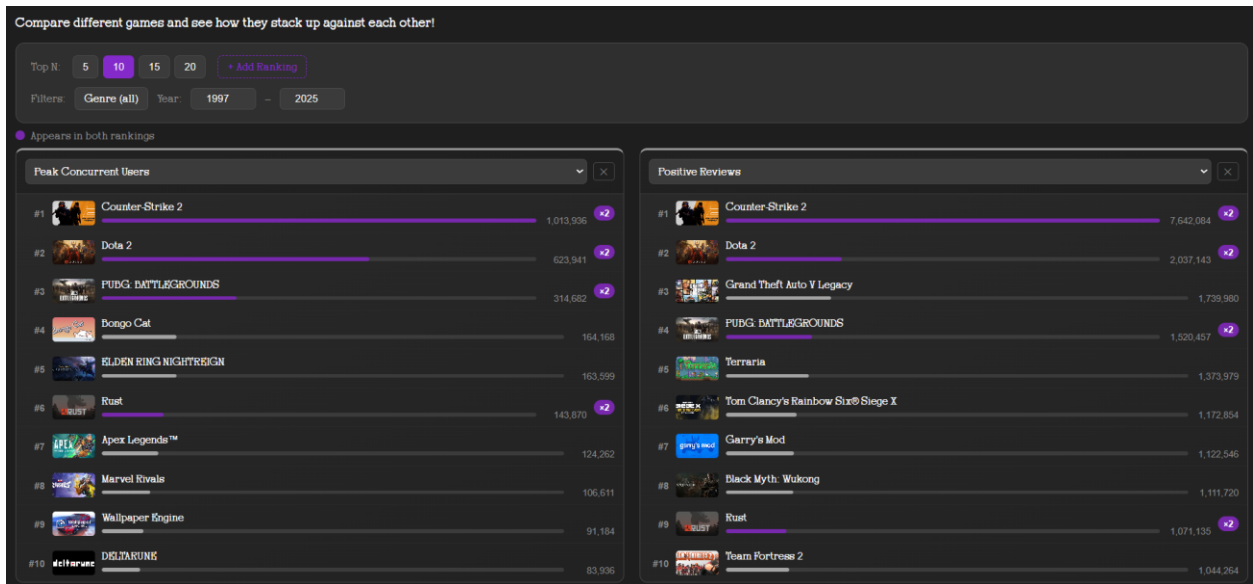
▪ Game Data Card:

This visualisation gathers all relevant information about a single game. We have focused on data that helps bring together the identity of a game. The game title contains a hyperlink to the game's Steam page. To compare the game's popularity against similar games, the user can select as many filters as they want (including developers, publishers, genres and tags). A dropdown ranks by peak CCU the first 50 games that have the same subset of filters.



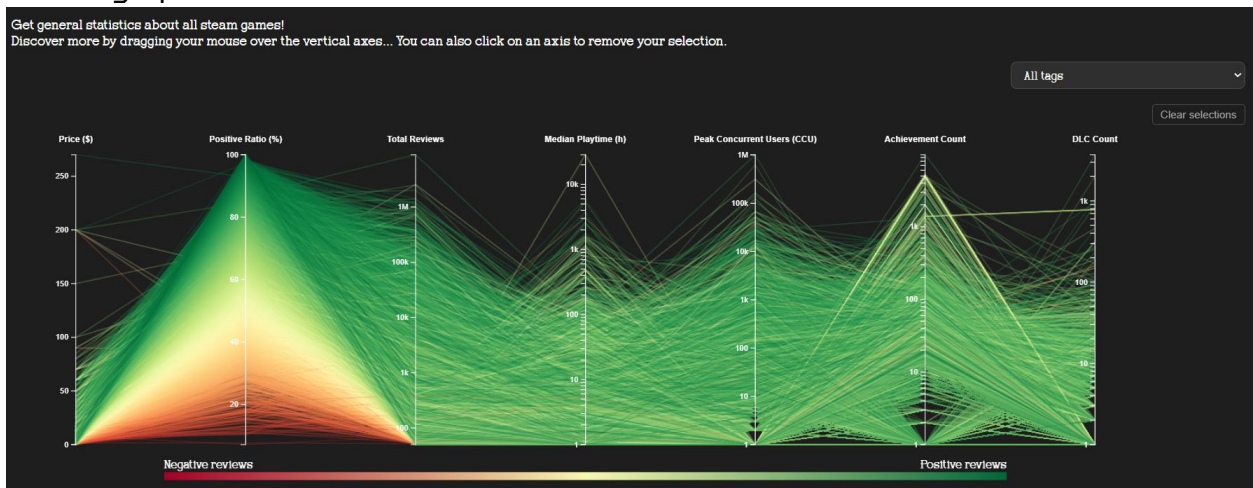
▪ Top Ranking:

These lists display the top games depending on different rankings that can be interactively chosen to fit the user's needs. A filter on genres allows the comparisons between more similar games, providing further insights. An additional color-coded visual effect highlights games that are in the top of multiple rankings. Clicking on the name of the game opens its Steam page.



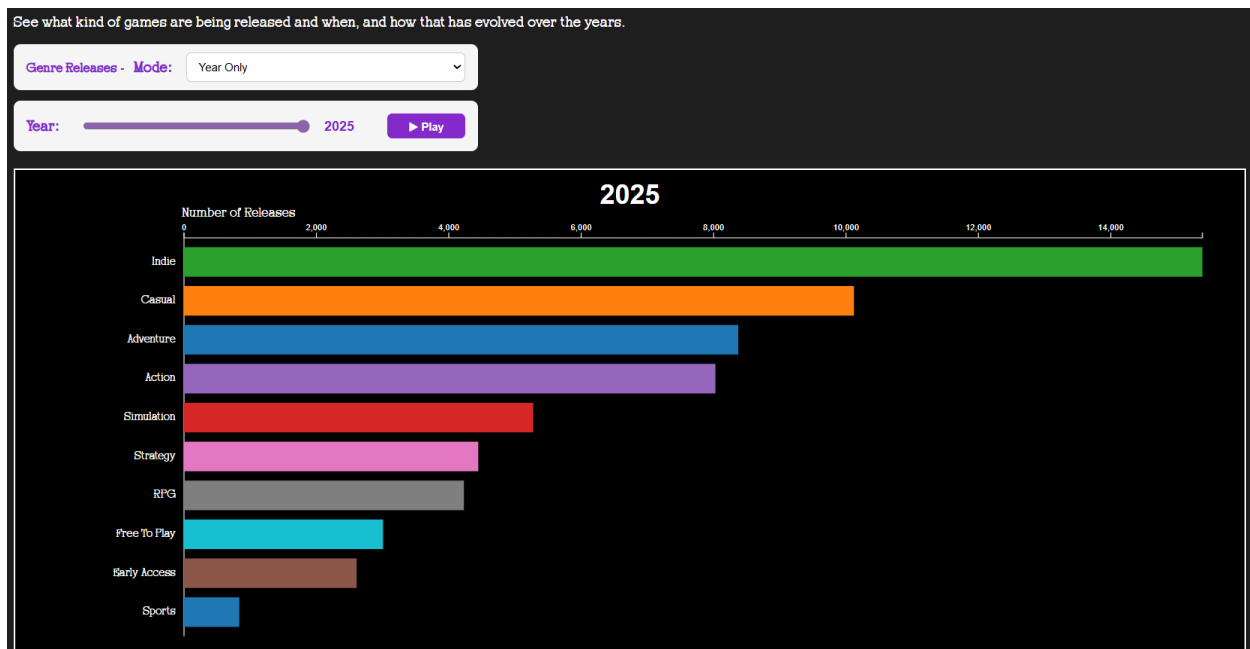
• Games Overview/Comparisons:

In this visualisation, we used a parallel-coordinates plot to represent different dimensions over the entire dataset of games. Various interactive features allow the user to narrow down the games visible on the graph by using a brush selection on the dimension axes and/or selecting a game tag to filter out different games. A hover functionality also shows the name of any specific game on the graph.



• Genre Release History:

This race bar shows the evolution of genre-specific releases throughout the years. It can be used on two modes, one that includes the releases of the displayed year only, and the other of all the games of up to that year. A slider allows the user to interact with the racing chart, and a pause/play button adds further control.



Challenges

The main challenge we faced during this project was the size of the dataset, which made it difficult to load and manipulate in the browser. The preprocessing pipeline helped to reduce the size of the dataset, but it still takes a significant amount of time to load and parse the data on the client side.

Even after the data is loaded, the number of entries still makes the rendering of the parallel coordinates plot extremely slow and unresponsive, as it requires drawing a polyline for each game in the dataset. To make the visualization more responsive, we had to implement a lot of optimizations, such as using a canvas element instead of SVG for rendering, caching and precomputing the positions of the polylines, and limiting the number of games that are rendered at a time during brushing: only a preview of the selected games is rendered during brushing, and the full rendering is only done at the end of the brushing. Even with these optimizations, the parallel coordinates plot is still not perfectly smooth, but it is now usable and allows exploring the relationships between multiple game attributes simultaneously.

The last issue we faced was with the scrapping process, which is very slow and can take more than 100 hours to complete. As we already had a pre-scrapped dataset at hand, we prioritized the development of the visualizations and sadly ran out of time to attach the most up-to-date dataset to the website before the deadline. Anyway, the scrapping process is perfectly functional and well-integrated in the project repository, and it can be tested for a few games to verify that the website works correctly.

Peer Assessment

Valentin:

- Integrated the "Steam Game Scraper" (by Martin Bustos) in the project repository
- Developed the preprocessing pipeline, which drastically reduces the size of the dataset and makes it more manageable for the website
- Performed an in-depth data exploration and analysis, which allowed us to find interesting ideas for visualizations.
- Configured the development environment, including its file structure, the Vite build tool, TypeScript, and the gh-pages package for deployment.
- Prototyped early versions of the parallel coordinates plot, and the single game view.
- Researched ways to optimize the rendering of the parallel coordinates plot, which was very slow due to the size of the dataset.
- Authored most of the documentation, including the README file and the different guides in the Documentation folder of the repository.

Lorie:

- Researched adequate datasets and visualization ideas
- Game comparator visualization implementation and optimization
- Bar race visualisation and optimisation
- Global optimisation of the implementations

Alice:

- Initial structure of the website with link navigation and styling
- Game card visualization with same-tag filter search
- Visualization and optimization of the parallel-coordinates plot
- Recording and editing the screencast

Common tasks:

- Process book writing

Resources

- [GitHub repository](#)
- [GitHub Pages deployment](#)
- [Screencast](#)